

Optimizarea deciziei de inlining în compilare prin tehnici de Machine Learning

*Inlining Compiler Optimization
Through Machine Learning Techniques*

Absolventă

Ingrid-Adriana Corobană

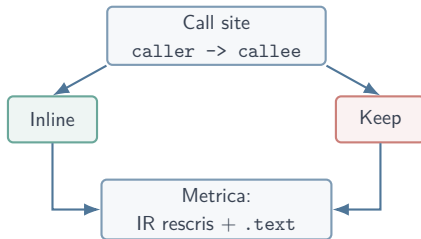
Profesor coordonator

Prof. Dr. Cristian Rusu

Lucrare de licență

Problema: o decizie locală cu efect global

- ▶ La fiecare apel, compilatorul decide: **inline** sau **keep**.
- ▶ Inlining-ul elimină overhead-ul apelului și poate dezvălui optimizări ulterioare.
- ▶ Poate duplica mult cod, crește binarul și schimbă oportunitățile de optimizare.



Încadrarea proiectului: fiecare call site devine o decizie a unei euristici, iar euristica este învățată din rezultate măsurate.

Când ajută și când strică inlining-ul

Bun: propagare de constante

```
string light_color(int c, int i) {  
    if (c == 0) return light_red(i);  
    if (c == 1) return light_green(i);  
    return light_blue(i);  
}  
  
string car_color(int i) {  
    int f = i;  
    return light_color(0, 2 + f);  
}
```

▼ inline

```
string car_color(int i) {  
    int f = i;  
    return light_red(2 + f);  
}
```

Apelul cu $c = 0$ face ramura relevantă vizibilă pentru optimizări.

Rău: expansiune recursivă

```
int sum(int x) {  
    if (x <= 0) return 0;  
    return x + sum(x - 1);  
}
```

▼ inline

```
int sum(int x) {  
    if (x <= 0) return 0;  
    return x + (x - 1) + sum(x - 2);  
    // apelul recursiv rămâne  
}
```

Rescrierea expune din nou aceeași structură, deci este nevoie de verificări de recursivitate și creștere.

Soluție

Ideea principală

- ▶ Extrag features pentru fiecare call site din LLVM IR.
- ▶ Rulez mai multe politici profesor și măsoar efectul lor după rescriere.
- ▶ Aleg cel mai bun profesor pe modul și construiesc un dataset de behavior cloning.
- ▶ Antrenez modele student care imită deciziile și le evaluez prin rescriere IR.

Contribuția mea

- ▶ Pipeline reproductibil end-to-end.
- ▶ Dataset sintetic plus surse reale.
- ▶ Politici profesor, selecție BC-Max și modele student.
- ▶ Evaluare cu artefacte inspectabile.

Comanda: `python3 scripts/run.py all`

Datele și reprezentarea call site-urilor

Surse

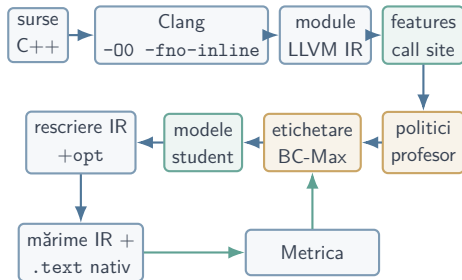
- ▶ Snapshot DCMTK:
config/tests/*.cc
- ▶ Module generate determinist pentru cazuri inline-positive și inline-negative
- ▶ Evaluare separată pe surse reale: 80 module, 9168 eşantioane train/test

Features

- ▶ structură caller/callee și cost estimat
- ▶ număr de utilizatori, înălțime call site, constante în parametri
- ▶ număr instrucțiuni, număr apeluri și semnal pentru recursivitate
- ▶ features globale: număr de noduri și muchii în graful de apeluri



Pipeline-ul complet

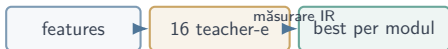


- Artefacte produse: features, etichete, metrice student, IR rescris, măsurători native și figuri.
- Metricul principal: reducerea numărului de instrucțiuni LLVM IR față de `never_inline`.

Politici profesor: cum sunt alese etichetele

De ce mai mulți profesori

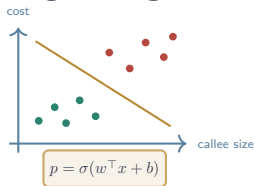
- ▶ acoperă extreme: `never_inline` și politici agresive;
- ▶ testează criterii diferite: dimensiune, cost, constante, recursivitate;
- ▶ BC-Max alege per modul profesorul cu IR rescris cel mai mic.



Teacher	Semnal folosit
<code>never_inline</code> <code>small_callee</code> <code>single_caller</code> <code>constant_argument</code>	baseline: păstrează toate apelurile funcții mici sau folosite o singură dată constante care pot declanșa simplifiări
<code>cost_budget</code> <code>growth_budget</code> <code>llvm_like_size</code> <code>loop_averse</code> <code>hot_leaf</code>	bugete pentru cost și creștere de cod
<code>aggressive_speed</code> <code>balanced_score</code> <code>benefit_cost_ratio</code>	control-flow, leaf calls, hotness proxy
<code>ml_linear_score</code> <code>bandit_ucb_proxy</code>	scoruri beneficiu-cost pe features
<code>rl_value_proxy</code> <code>greedy_ir_size</code>	proxy-uri pentru decizii secvențiale alese după IR rescris

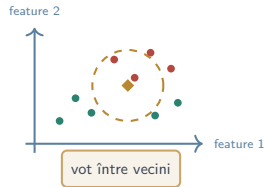
Modele student

Logistic regression



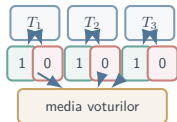
Un scor linear peste vectorul de 16 features.

KNN



Decizia vine din call site-uri similare.

Forest



Media voturilor stabilizează decizia.

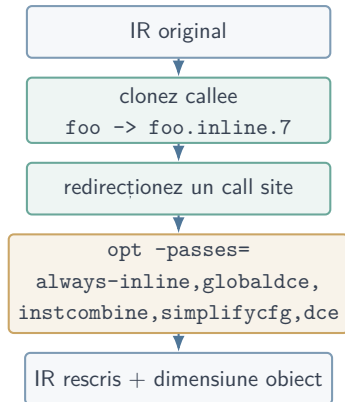
MLP



Straturile ascunde modelează interacțiuni între features.

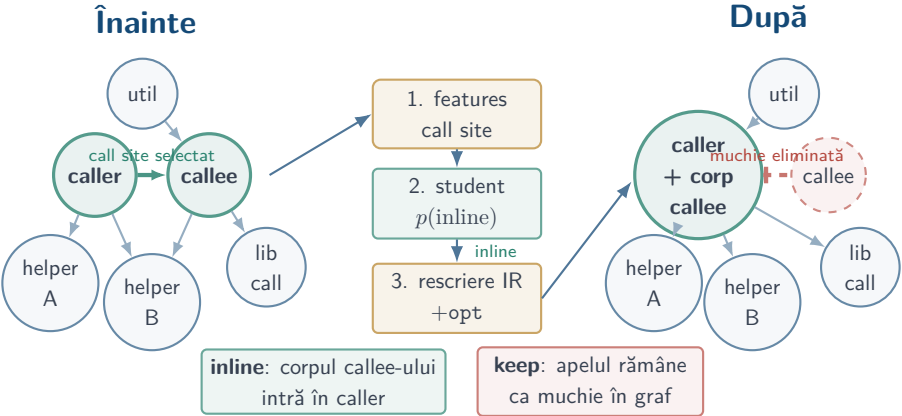
Cum se face rescrierea IR

1. Citesc LLVM IR textual.
2. Elimin `noinline`: funcția poate fi inlined.
3. Clonez callee-ul selectat cu nume unic.
4. Marchez clona cu `alwaysinline`.
5. Redirecționez numai apelul selectat.
6. Rulez LLVM `opt`: aplică pass-urile de optimizare pe IR.

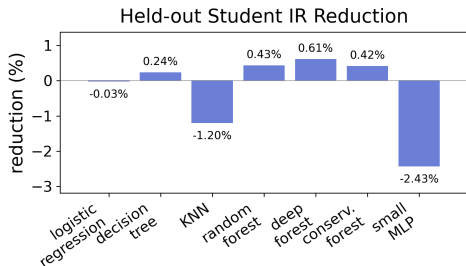


`always-inline`: materializează apelurile marcate; `globaldce`: elimină funcții/globale nefolosite; `instcombine`: simplifică instrucțiuni; `simplifycfg`: curăță control-flow-ul; `dce`: elimină cod mort.

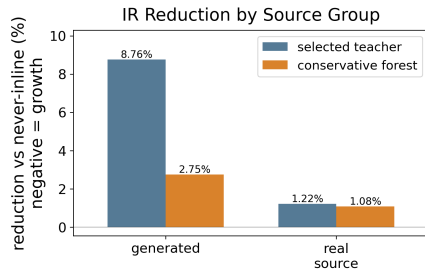
Exemplu vizual: ce se schimbă în graful de apeluri



Rezultate



- ▶ 10.394 call site-uri analizate.
- ▶ Held-out: cel mai bun model obține **0,61%** reducere IR.
- ▶ Rulare principală: **conservative forest** are **1,18%** reducere totală IR.



- ▶ Selected-teacher pe held-out: **1,10%** reducere IR.
- ▶ Surse reale: **deep forest** obține **1,08%** reducere totală IR.
- ▶ Evaluarea pune accent pe rescrierea IR și folosește match-ul de etichete ca diagnostic.

Comandă

```
python3 scripts/run.py demo
```

Ce arată

- ▶ compilează module C++ generate în LLVM IR;
- ▶ extrage features pentru call site-uri;
- ▶ aplică decizii prin același mecanism clone-and-opt;
- ▶ scrie decizii (0=keep, 1=inline) și IR rescris în out/demo/.

```
out > demo > ir > * generated_000.ll
{
  "source_filename": "generated_000.ll",
  "target datalayout": "e-m64-p270:32:32-p271:32:32-p272:64:64-164:64-128+32:64-5120-fn32",
  "target triple": "arm64-apple-macosx15.0.0"
}

OUTPUT PROBLEMS PORTS TERMINAL

python3 scripts/run_delta_signal.py demo
IngridCorobana@Ingrid's-MacBook-Pro thesis % cd /Users/IngridCorobana/Desktop/optimizingInlining
$LLVM_CONFIG=opt/llvmconfig python3 scripts/run.py demo

demo
-----
compiled demo modules: 4
failed demo modules: 0
demo callsites: 80
inline decisions: 0
order: sorted .ll files => LLVM function definition order => call-line order
generated_000.ll: callsite7, inline8, local calls 17 -> 9, instr 291 -> 291
[0] pome_000_entry -> -2.15pome_000_tiny_01: inline
[01] pome_000_entry -> -2.15pome_000_tiny_11: inline
[02] pome_000_entry -> -2.15pome_000_tiny_21: inline
[03] pome_000_entry -> -2.15pome_000_tiny_31: inline
[04] pome_000_entry -> -2.15pome_000_tiny_41: inline
generated_001.ll: callsite7, inline8, local calls 17 -> 17, instr 439 -> 317
[0] image_001_entry -> -2.18image_001_branch_11: keep
[01] image_001_entry -> -2.18image_001_branch_11: keep
[02] image_001_entry -> -2.18image_001_branch_21: keep
[03] image_001_entry -> -2.18image_001_branch_31: keep
[04] image_001_entry -> -2.18image_001_branch_41: keep
generated_002.ll: callsite7, inline8, local calls 17 -> 17, instr 469 -> 469
[0] packet_002_entry -> -2.19packet_002_medium_01: keep
[01] packet_002_entry -> -2.19packet_002_medium_11: keep
[02] packet_002_entry -> -2.19packet_002_medium_21: keep
[03] packet_002_entry -> -2.19packet_002_medium_31: keep
[04] packet_002_entry -> -2.19packet_002_medium_41: keep
generated_003.ll: callsite7, inline8, local calls 17 -> 17, instr 235 -> 235
[0] matrix_003_entry -> -2.16matrix_003_large_01: keep
```

Demo: complete_optimizing_inlining_demo.mp4

Limitări și concluzii

Limitări

- ▶ prototip peste LLVM IR, cu integrare LLVM pass ca pas următor;
- ▶ rescriere textuală de IR, cu aceeași selecție deterministă a call site-urilor;
- ▶ metrice pe instrucțiuni IR și obiect .text;
- ▶ problemă secvențială aproximată offline: deciziile sunt alese pe seturi fixe de call site-uri.

Concluzii

- ▶ Am demonstrat un workflow reproductibil pentru inlining ghidat de ML.
- ▶ Contribuția teoretică: formularea inlining-ului ca problemă offline de imitare, cu profesori euristici și selecție per modul.
- ▶ Contribuția practică: legătura dintre decizii ML și artefacte de compilator inspectabile.
- ▶ Pași următori: benchmark-uri mai mari, profesori mai puternici, integrare ca LLVM pass.